

Как да напишем student.cpp

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · вграден клас и наследник

Какво ще направим

Ще напишем програма с **три класа**, свързани по два различни начина. Това е основната идея на темата: има два начина един клас да използва друг и те не са взаимозаменяеми.

Връзка	Как се чете	Как изглежда в кода
Вграждане (композиция)	Студентът има адрес	<code>CAddress m_address;</code> — обикновена член-променлива
Наследяване (производност)	Студентът е личност	<code>class CStudent : public CPerson</code>

Проверката е проста. Ако можете да кажете „X **е** Y“ — наследяване. Ако можете да кажете само „X **има** Y“ — вграждане. Студентът има адрес, но не е адрес. Студентът е личност, а не има личност.

Освен това ще напишем за първи път **копиращ конструктор** — специалната функция, която се вика, когато един обект се създава като копие на друг.

План на работата

1. Нов проект и скелет
2. Класът `CPerson` от тема 1 — базата
3. Класът `CAddress` — данните (изискване I.1–3)
4. Трите конструктора на `CAddress` (изискване I, публични 1–3)
5. Функция за извеждане `Output` (изискване I.4)
6. Класът `CStudent` — наследяване (изискване II)
7. Конструкторите на наследника (изискване II.1–3)
8. Извеждане и достъп до членовете (изискване II.4–5)
9. Главна функция (изискване III)

След всяка стъпка кодът се компилира. Ако сгрешите, ще знаете в коя стъпка, вместо да търсите грешката в 400 реда наведнъж.

1 Нов проект и скелет

Във Visual Studio: **File** → **New** → **Project** → **Empty Project**, име например `Tema2_Student`. В **Solution Explorer** щракнете с десен бутон върху **Source Files** → **Add** → **New Item** → **C++ File (.cpp)** и напишете име `student.cpp`.

Не добавяйте този файл към проекта от тема 1. Там вече има функция `main`. Два файла с `main` в един проект дават грешка `LNK2005`. Всяка задача — отделен проект.

НАПИШЕТЕ ТОВА В ПРАЗНИЯ ФАЙЛ

```
#include <cctype>      // isdigit
#include <cstdio>      // getchar
#include <iostream>
#include <string>
using namespace std;

int main()
{
    cout << "Програмата работи!" << endl;

    getchar();          // задържа прозореца отворен
    return 0;
}
```

Проверка: `Ctrl + Shift + B` (Build) → 1 succeeded, 0 failed.

2 Класът `CPerson` от тема 1 — базата

Условието казва: клас `CStudent` наследник на `CPerson`, създаден от предното упражнение. Затова първо трябва да имаме `CPerson` в този файл.

Вземете класа `CPerson2` от тема 1, преименувайте го на `CPerson` и направете две промени.

ПРОМЯНА 1: `PRIVATE` СТАВА `PROTECTED`

```
class CPerson
{
protected:          // беше private
    string m_strName;
    Gender m_gender;
    string m_strEGN;
    ...
}
```

Спецификатор	Вижда се от самия клас	Вижда се от наследник	Вижда се отвън
<code>private</code>	да	не	не
<code>protected</code>	да	да	не
<code>public</code>	да	да	да

Защо е нужно? `CStudent` ще иска да ползва `m_strName` направо. При `private` това е забранено дори за наследника — ще получите `C2248: cannot access private member`. При `protected` е разрешено, а за външния свят данните остават също толкова скрити, колкото при `private`.

Кое е по-добро в реален проект? `private` плюс публични `Get.../Set...`, защото тогава базовият клас може да смени вътрешното си устройство, без да счупи наследниците. Тук използваме `protected`, защото темата е точно за това какво вижда наследникът.

ПРОМЯНА 2: ДОБАВЯМЕ КОПИРАЩ КОНСТРУКТОР И `OUTPUT`

```
public:
    CPerson();
    CPerson(const string& strName, const string& strGender, const string& strEGN);
    CPerson(const CPerson& pers);                // копиращ конструктор

    ostream& Output(ostream& toStream) const;    // ще го ползва и CStudent
```

```
// Копиращ конструктор: създава нов обект като точно копие на подадения.
CPerson::CPerson(const CPerson& pers)
    : m_strName(pers.m_strName), m_gender(pers.m_gender), m_strEGN(pers.m_strEGN)
{
}

ostream& CPerson::Output(ostream& toStream) const
{
    toStream << m_strName << " (" << genderToString() << "), ЕГН: "
              << (m_strEGN.empty() ? string("няма") : m_strEGN);
    return toStream;
}
```

Останалото — `enum Gender`, `stringToGender`, `genderToString`, `isValid`, `SetEGN` — се пренася без промяна. Пълният текст е в листинга накрая.

Проверка: компилирайте. `main` още не използва класа — това е нормално.

3 Класът `CAddress` — данните

Условието изисква три частни низа. Започваме само с данните.

ДОБАВЕТЕ НАД ФУНКЦИЯТА `MAIN`

```
class CAddress
{
private:
    string m_strStreet;    // 1. улица, напр. "Studentska #1"
    string m_strPostCode;  // 2. пощенски код, напр. "9010"
    string m_strCity;      // 3. град, напр. "Varna"
};
```

Защо пощенският код е низ, а не число? Защото с него не се смята. Освен това `9010` като число губи водещата нула, ако кодът е `0900`. Правилото: щом не събирате и не умножавате стойността, тя е текст.

Най-честата грешка тук: пропусната точка и запетая след затварящата скоба. Пише се `};`, а не само `}`. Visual Studio дава подвеждащо `C2143: syntax error`, което сочи някъде другаде.

4 Трите конструктора на `CAddress`

Условието изисква три конструктора. Ще ги обявим в класа и ще ги напишем след него — така класът остава четим като съдържание, а телата са отделно.

ДОБАВЕТЕ В КЛАСА, СЛЕД ЧЛЕН-ПРОМЕНЛИВИТЕ

```
public:
    CAddress(); // 1. по подразбиране
    CAddress(const string& strStreet, // 2. експлицитен
              const string& strPostCode,
              const string& strCity);
    CAddress(const CAddress& addr); // 3. копиращ
```

НАПИШЕТЕ СЛЕД ЗАТВАРЯЩАТА СКОБА НА КЛАСА

```
// 1. Конструктор по подразбиране
CAddress::CAddress()
    : m_strStreet("(няма улица)", m_strPostCode("0000"), m_strCity("(няма град)")
{
}

// 2. Експлицитен конструктор
CAddress::CAddress(const string& strStreet, const string& strPostCode, const string& strCity)
    : m_strStreet(strStreet), m_strPostCode(strPostCode), m_strCity(strCity)
{
}

// 3. Копиращ конструктор
CAddress::CAddress(const CAddress& addr)
    : m_strStreet(addr.m_strStreet),
      m_strPostCode(addr.m_strPostCode),
      m_strCity(addr.m_strCity)
{
}
```

КАКВО Е ДВОЕТОЧИЕТО СЛЕД ИМЕТО

Това е **списък за инициализация**. Той дава стойности на членовете *в момента на създаването им*, преди тялото `{ }` да се изпълни. Алтернативата

```
CAddress::CAddress() { m_strStreet = "(няма улица)"; }
```

първо създава `m_strStreet` празен, а после му присвоява стойност — две действия вместо едно. За `string` разликата е малка, но за член, който няма конструктор по подразбиране, или за `const` член,

списъкът за инициализация е единственият възможен начин.

ТРИТЕ КОНСТРУКТОРА ЕДИН ДО ДРУГ

Конструктор	Кога се вика	Пример
по подразбиране	при създаване без аргументи	<code>CAddress addr1;</code>
експлицитен	при създаване с аргументи	<code>CAddress addr2("Studentska #1", "9010", "Varna");</code>
копиращ	при създаване от съществуващ обект	<code>CAddress addr3(addr2);</code>

Защо параметърът на копиращия е `const CAddress&` ? Двете части са задължителни по различни причини.

`&` (референция) — ако параметърът беше по стойност (`CAddress addr`), то за да се предаде, трябва да се направи копие... с копиращия конструктор... който отново трябва да предаде параметъра... Безкрайна рекурсия. Компиляторът дори не я допуска: `C2652: illegal copy constructor`.

`const` — обещание, че оригиналът няма да бъде променен. Без него не можете да копирате константен обект.

Ако не напишете копиращ конструктор, класът пак ще се копира. Компиляторът генерира свой, който копира всеки член поотделно. За `CAddress` резултатът е същият — затова тук пишем копиращия конструктор, за да се види как работи, а не защото без него нещо ще се счупи. Има случаи, в които генерираният е **опасен** — когато класът държи указател към динамична памет. Това е темата на следващото упражнение.

Проверка: компилирайте. Ако получите `C2065: m_strStreet: undeclared identifier`, значи телата на конструкторите са написани *преди* класа. Преместете ги след него.

5 Функция за извеждане `Output`

Условието задава точна сигнатура: `ostream& Output(ostream& toStream)`.

ОБЯВЕТЕ В КЛАСА

```
ostream& Output(ostream& toStream) const;
```

НАПИШЕТЕ СЛЕД КЛАСА

```
ostream& CAddress::Output(ostream& toStream) const
{
    toStream << m_strCity << " " << m_strPostCode << ", " << m_strStreet;
    return toStream;
}
```

ТРИТЕ РЕШЕНИЯ В ТЕЗИ ЧЕТИРИ РЕДА

- 1 **Параметърът е `ostream&`, а не `cout`.** Така една и съща функция извежда на конзолата (`addr.Output(cout)`) и във файл (`addr.Output(oFile)`). Класът не знае къде отива текстът — и това е добре.
- 2 **Връща се `ostream&` — самият поток, не копие.** Потоките не могат да се копират. Затова типът на връщане задължително е референция. Това позволява верижно писане: `addr.Output(cout) << endl;`
- 3 **Няма `endl` вътре.** Функцията извежда своите данни и спира. Кой ще мине на нов ред решава извикващият. Ако сложите `endl` вътре, никога няма да можете да отпечтатате адрес в средата на ред — а точно това прави `CStudent::Output` в стъпка 8.

Сега вие: напишете по същия образец шестте функции за четене и презапис — `GetStreet`, `GetPostCode`, `GetCity` (връщат `const string&`, маркирани са `const`) и `SetStreet`, `SetPostCode`, `SetCity` (приемат `const string&`, връщат `void`). Всяка е един ред. Сравнете с листинга накрая.

ПРОБВАЙТЕ В MAIN

```
CAddress addr2("Studentska #1", "9010", "Varna");
addr2.Output(cout) << endl;
```

Проверка: стартирайте. Трябва да се изведе `Varna 9010, Studentska #1`. Изтрийте временните редове.

6 Класът `CStudent` — наследяване

Сега идва същината на темата. Един ред прави `CStudent` наследник.

ДОБАВЕТЕ СЛЕД КЛАСА `CADDRESS`

```
class CStudent : public CPerson
{
private:
    string m_strFN;           // 1. факултетен номер
    CAddress m_address;       // 2. ВГРАДЕН обект от класа CAddress
};
```

КАКВО ТОЧНО ОЗНАЧАВА ДВОЕТОЧИЕТО

`: public CPerson` казва на компилатора: *всеки обект от тип `CStudent` съдържа вътре в себе си цял обект от тип `CPerson`*. Тоест `CStudent` получава наготово:

- член-променливите `m_strName`, `m_gender`, `m_strEGN` (достъпни, защото са `protected`);
- всички публични функции — `SetName`, `GetName`, `SetGender`, `SetEGN`, `genderToString`, `isValid`...

Не се пише нито ред повече, за да ги има. `stud.SetName("Иван")` работи веднага.

ДВАТА ВИДА ВРЪЗКА В ЕДИН КЛАС

```
: public CPerson        CAddress m_address;
```

Как се чете	студентът е личност	студентът има адрес
Достъп до членовете	директно: <code>m_strName</code>	през обекта: <code>m_address.GetCity()</code>
Може ли <code>CPerson* p = &stud;</code>	да	не

Защо `public CPerson`, а не само `CPerson`? Ако пропуснете думата `public`, при `class` наследяването по подразбиране е `private`. Тогава всички наследени публични функции стават частни и `stud.SetName(...)` дава C2247: `CPerson::SetName not accessible because CStudent uses private to inherit`. Почти винаги се иска `public`.

7 Конструкторите на наследника

Условието иска четири: по подразбиране, два експлицитни и копиращ.

ОБЯВЕТЕ В КЛАСА

```
public:
    CStudent();
    CStudent(const string& strName, const CAddress& addr, const string& strEGN);
    CStudent(const string& strName, const CAddress& addr, const string& strEGN,
              const string& strFN);
    CStudent(const CStudent& stud);
```

ПЪРВО ГРЕШНИЯТ НАЧИН

```
// НЕ пишете това!
CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN)
{
    m_strName = strName;           // работи, но...
    m_strEGN  = strEGN;           // заобикаля SetEGN и валидизатора!
    m_address = addr;
}
```

Проблемът не е че не се компилира — компилира се. Проблемът е, че базовият клас вече е бил построен *преди* тялото да започне: конструкторът по подразбиране `CPerson()` е сложил „Неизвестен“, а сега ние го презаписваме. Двойна работа. И по-лошо — записът в `m_strEGN` направо заобикаля проверката за валидност.

НАПИШЕТЕ СЛЕД КЛАСА

```
// 1. Конструктор по подразбиране
CStudent::CStudent()
    : CPerson(), m_strFN("(няма)"), m_address()
{
}

// 2. Експлицитен конструктор по име, адрес и ЕГН
CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN)
    : CPerson(strName, "неопределен", strEGN), m_strFN("(няма)"), m_address(addr)
{
}
```

```
// 2. Експлицитен конструктор по име, адрес, ЕГН и фак. номер
CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN,
                    const string& strFN)
    : CPerson(strName, "неопределен", strEGN), m_strFN(strFN), m_address(addr)
{
}

// 3. Копиращ конструктор
CStudent::CStudent(const CStudent& stud)
    : CPerson(stud), m_strFN(stud.m_strFN), m_address(stud.m_address)
{
}
```

РЕДЪТ НА ИЗПЪЛНЕНИЕ

При `CStudent stud3("Елена", addr2, "1111111104", "21621301");` се случва следното, точно в този ред:

- 1 `CPerson(strName, "неопределен", strEGN)` — първо се построява **базовата част**. Винаги първо базата.
- 2 `m_strFN(strFN)` — после собствените членове, **по реда на обявяването им в класа**, а не по реда в списъка.
- 3 `m_address(addr)` — вграденият обект се построява с копиращия конструктор на `CAddress`.
- 4 тялото `{ }` — тук е празно, защото всичко вече е готово.

Пишете членовете в списъка в реда, в който са обявени в класа. Компиляторът инициализира по реда на обявяване независимо какво пише в списъка. Ако редът се разминава, Visual Studio дава предупреждение `C5038`. Не е грешка, но ако един член използва друг при инициализацията, разминаването води до четене на неинициализирана стойност.

Защо конструкторите не приемат пол? Условието изброява точно „по име, адрес и ЕГН“ и „по име, адрес, ЕГН и факултетен номер“. Затова подаваме "неопределен", а полът се задава после с наследената `SetGender`. Да добавите четвърти параметър за пол не може — тогава `CStudent(name, addr, egn, "мъж")` и `CStudent(name, addr, egn, "21621301")` стават неразличими: и двата са четири низа. Компиляторът би отказал с `C2668: ambiguous call`.

КОПИРАЩИЯТ КОНСТРУКТОР РЕД ПО РЕД

- | | |
|--|---|
| <code>CPerson(stud)</code> | Копира наследената част. Подаваме <code>stud</code> (тип <code>CStudent</code>) там, където се очаква <code>const CPerson&</code> — позволено е, защото студентът е личност. Взема се само личностната част. |
| <code>m_strFN(stud.m_strFN)</code> | Копира собствения член. |
| <code>m_address(stud.m_address)</code> | Копира вградения обект — вика се копиращият конструктор на <code>CAddress</code> . |

Проверка: компилирайте. Ако получите `C2512: CPerson: no appropriate default constructor available`, значи някой конструктор на `CStudent` не е извикал базовия изрично и `CPerson` няма конструктор по подразбиране. Тук го има, така че грешката по-скоро значи печатна грешка в името.

8 Извеждане и достъп до членовете

ОБЯВЕТЕ В КЛАСА

```
const string& GetFN() const;
const CAddress& GetAddress() const;
void SetFN(const string& strFN);
void SetAddress(const CAddress& addr);

ostream& Output(ostream& toStream) const;
```

НАПИШЕТЕ СЛЕД КЛАСА

```
const string& CStudent::GetFN() const { return m_strFN; }
const CAddress& CStudent::GetAddress() const { return m_address; }

void CStudent::SetFN(const string& strFN) { m_strFN = strFN; }
void CStudent::SetAddress(const CAddress& addr) { m_address = addr; }

ostream& CStudent::Output(ostream& toStream) const
{
    CPerson::Output(toStream); // 1. наследената част
    toStream << ", ф.н.: " << m_strFN << ", адрес: ";
    m_address.Output(toStream); // 2. вградената част
    return toStream;
}
```

ДВАТА НАЧИНА НА ИЗВИКВАНЕ В ЕДНА ФУНКЦИЯ

`CPerson::Output(toStream)`

Наследената функция. Пише се с `::` и име на базовия клас, защото `CStudent` има собствена `Output` и трябва да се каже коя точно се иска. Без `CPerson::` функцията ще извика **себе си** — безкрайна рекурсия до препълване на стека и срив.

`m_address.Output(toStream)`

Вграденият обект. Пише се с точка, като всяко обръщение към член-променлива.

Това е най-опасната грешка в цялата тема. Ако напишете само `Output(toStream);` вместо `CPerson::Output(toStream);`, кодът се компилира без нито едно предупреждение, а програмата се срива при първото извикване със `Stack overflow`. Симптомът не наемква за причината — запомнете го отсега.

Защо `GetAddress` връща `const CAddress&`? Референция — за да не се прави ненужно копие на целия обект. `const` — за да не може отвън да се пише `stud.GetAddress().SetCity("Ruse")` и така да се заобиколи `SetAddress`. Без `const` частният член би бил напълно отворен за промяна.

9 Главна функция

Условието иска обекти „от различни видове“ и тест на функциите — за двата класа поотделно. Добавяйте по една група и компилирайте след всяка.

9.1 Трите конструктора на CAddress

```
cout << "==== A. Класът CAddress =====" << endl;

CAddress addr1;                                     // подразбиращ се
CAddress addr2("Studentska #1", "9010", "Varna");    // експлицитен
CAddress addr3(addr2);                               // копиращ

cout << "  addr1 (подразбиращ се): "; addr1.Output(cout) << endl;
cout << "  addr2 (експлицитен):    "; addr2.Output(cout) << endl;
cout << "  addr3 (копие на addr2): "; addr3.Output(cout) << endl;
```

9.2 Копието е независимо

```
cout << "---- Тест: копието е независимо ----" << endl;
addr3.SetStreet("Studentska #2");
cout << "  addr2 след промяна на addr3: "; addr2.Output(cout) << endl;
cout << "  addr3 след промяна:          "; addr3.Output(cout) << endl;
```

Този тест изглежда тривиален, но не е. Той доказва, че копиращият конструктор прави **отделен** обект, а не второ име на същия. Сравнете с изхода при референция в 9.5 — там промяната се вижда и през двете имена.

9.3 Масив от обекти

```
CAddress arrAddr[3] = {
    CAddress("Studentska #1", "9010", "Varna"),
    CAddress("Tsar Osvoboditel #12", "1000", "Sofia"),
    CAddress()
};
const int nAddrCount = sizeof(arrAddr) / sizeof(arrAddr[0]);
for (int i = 0; i < nAddrCount; i++)
{
    cout << "  [" << i << "] ";
    arrAddr[i].Output(cout) << endl;
}
```

9.4 Указател и динамична памет

```
CAddress* pAddr = new CAddress("Slivnitsa #33", "9002", "Varna");
cout << "  ";
pAddr->Output(cout) << endl;
delete pAddr;
pAddr = nullptr;
```

9.5 Референция и константен обект

```
CAddress& refAddr = addr1;
refAddr.SetCity("Burgas");
refAddr.SetPostCode("8000");
cout << "  addr1 след промяна през refAddr: "; addr1.Output(cout) << endl;

const CAddress cAddr("Studentska #1", "9010", "Varna");
```

```
cout << "    град: " << cAddr.GetCity() << ", код: " << cAddr.GetPostCode() << endl;
// cAddr.SetCity("Ruse");           // грешка: обектът е константен
```

Задача за проверка: разкоментирайте последния ред и компилирайте. Очаква се C2662 — върху константен обект може да се вика само функция, маркирана с `const`. Върнете реда като коментар.

9.6 Четирите конструктора на CStudent

```
cout << endl << "===== Б. Класът CStudent =====" << endl;

CStudent stud1;                                     // подразбиращ се
CStudent stud2("Иван Петров", addr2, "111111110");   // име, адрес, ЕГН
CStudent stud3("Елена Димитрова", addr2, "1111111104", "21621301");
CStudent stud4(stud3);                               // копиращ

cout << "    stud1: "; stud1.Output(cout) << endl;
cout << "    stud2: "; stud2.Output(cout) << endl;
cout << "    stud3: "; stud3.Output(cout) << endl;
cout << "    stud4: "; stud4.Output(cout) << endl;
```

9.7 Наследените функции работят върху наследника

```
stud2.SetGender("мъж");                             // функция на CPerson
stud3.SetGender(FEMALE);                             // предефинираната версия
stud2.SetFN("21621300");                             // функция на CStudent
cout << "    GetName() на stud3: " << stud3.GetName() << endl;
cout << "    genderToString() на stud3: " << stud3.genderToString() << endl;
```

Това е целият смисъл на наследяването: `SetGender`, `GetName` и `genderToString` не са писани никъде в `CStudent`, но работят.

9.8 Указател към базовия клас

```
CPerson* pPerson = &stud3;                           // CStudent E CPerson
cout << "    през CPerson*: ";
pPerson->Output(cout) << endl;                         // вика CPerson::Output!
```

Разгледайте внимателно изхода на този ред. Отпечатва се само Елена Димитрова (жена), ЕГН: 1111111104 — без факултетен номер и без адрес. Обектът си е `CStudent`, но извиква се `CPerson::Output`.

Причината: компилаторът избира коя функция да извика според **типа на указателя** (`CPerson*`), а не според типа на обекта. Нарича се *ранно свързване*. Решението е една дума — `virtual` — и е тема на упражнение 5. Дотогава запомнете симптома: през указател към базов клас се вижда само базовата част.

9.9 Останалите видове променливи

Масив от студенти, указател с `new / delete`, референция, константен обект и достъп до вградения обект през `GetAddress().GetCity()` — по същия образец като 9.3–9.5. Пълният текст е в листинга.

Компилиране и стартиране

Вариант А — през средата на Visual Studio

Клавиши	Действие
<code>Ctrl</code> + <code>Shift</code> + <code>B</code>	Build — само компилира. Резултатът е в прозореца Output.
<code>F5</code>	Start Debugging — компилира и стартира с дебъгер. Прозорецът се затваря веднага след <code>main</code> , затова е нужен <code>getchar()</code> .
<code>Ctrl</code> + <code>F5</code>	Start Without Debugging — Visual Studio сам задържа прозореца.
<code>F9</code>	Точка на прекъсване. Поставете я на първия ред от конструктор и с <code>F11</code> (Step Into) проследете реда на изпълнение от стъпка 7.

Вариант Б — от команден ред

Компилаторът `cl` не работи в обикновен `cmd`. Отворете **Developer Command Prompt for VS**.

```
cl /EHsc /std:c++17 /utf-8 student.cpp
student.exe
```

Флаг	Значение
<code>/EHsc</code>	Включва обработката на изключения. Без него — предупреждение <code>C4530</code> .
<code>/std:c++17</code>	Задава стандарта на езика.
<code>/utf-8</code>	Файлт е в UTF-8. Задължителен при кирилица.

Кирилица под Windows

Ако вместо български букви виждате `???`, проверете трите неща в този ред:

- Файлт трябва да е в UTF-8.** File → Save As → стрелката до Save → Save with Encoding → *Unicode (UTF-8 with signature)* — *Codepage 65001*.
- Компилаторът трябва да знае за UTF-8.** Флаг `/utf-8`, или Project → Properties → C/C++ → Command Line → Additional Options.
- Конзолата трябва да е в UTF-8.** Изпълнете `chcp 65001` преди стартиране, или добавете като първи ред в `main`:

```
#include <cstdlib> // горе при останалите include-ове
system("chcp 65001 > nul"); // първи ред в main
```

Ако след `chcp 65001` виждате квадратчета — сменете шрифта на конзолата: десен бутон върху заглавната лента → Properties → Font → Consolas или Lucida Console.

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 2: Производност и наследяване. Вградени класове.
// CAddress (вграден клас) + CStudent (наследник на CPerson от тема 1).

#include <cctype>      // isdigit
#include <cstdio>      // getchar
#include <iostream>
#include <string>
using namespace std;

// ===== Изброим тип от тема 1 =====
enum Gender
{
    MALE,           // мъж
    FEMALE,         // жена
    UNKNOWN         // неопределен
};

// =====
// Класът CPerson от тема 1 – БАЗОВ клас за CStudent
// =====
class CPerson
{
    // protected вместо private: наследниците виждат данните,
    // но за външния свят те остават скрити както при private.
protected:
    string m_strName;
    Gender m_gender;
    string m_strEGN;

public:
    CPerson();
    CPerson(const string& strName, const string& strGender, const string& strEGN);
    CPerson(const CPerson& pers);           // копиращ конструктор

    void SetName(const string& strName);
    void SetGender(Gender gender);
    void SetGender(const string& strGender);
    bool SetEGN(const string& strEGN);

    const string& GetName() const;
    Gender GetGender() const;
    const string& GetEGN() const;

    Gender stringToGender(const string& strGender) const;
    string genderToString() const;
    bool isValid(const string& strEGN);

    ostream& Output(ostream& toStream) const;
};

// ----- Конструктори -----
CPerson::CPerson()
```

```

        : m_strName("Неизвестен"), m_gender(UNKNOWN), m_strEGN("")
    {
    }

CPerson::CPerson(const string& strName, const string& strGender, const string& strEGN)
    : m_strName(strName), m_gender(UNKNOWN), m_strEGN("")
{
    m_gender = stringToGender(strGender);
    SetEGN(strEGN);
}

// Копиращ конструктор: създава нов обект като точно копие на подадения.
CPerson::CPerson(const CPerson& pers)
    : m_strName(pers.m_strName), m_gender(pers.m_gender), m_strEGN(pers.m_strEGN)
{
}

// ----- Презапис -----
void CPerson::SetName(const string& strName)      { m_strName = strName; }
void CPerson::SetGender(Gender gender)           { m_gender = gender; }
void CPerson::SetGender(const string& strGender) { m_gender = stringToGender(strGender); }

bool CPerson::SetEGN(const string& strEGN)
{
    if (!IsValid(strEGN))
        return false;
    m_strEGN = strEGN;
    return true;
}

// ----- Четене -----
const string& CPerson::GetName() const { return m_strName; }
Gender CPerson::GetGender() const { return m_gender; }
const string& CPerson::GetEGN() const { return m_strEGN; }

// ----- Преобразуване на изброимия тип -----
Gender CPerson::stringToGender(const string& strGender) const
{
    if (strGender == "мъж" || strGender == "м" || strGender == "male")
        return MALE;
    if (strGender == "жена" || strGender == "ж" || strGender == "female")
        return FEMALE;
    return UNKNOWN;
}

string CPerson::genderToString() const
{
    switch (m_gender)
    {
        case MALE:    return "мъж";
        case FEMALE:  return "жена";
        default:      return "неопределен";
    }
}

// ----- Валидизатор на ЕГН (алгоритъм на ГРАО) -----
bool CPerson::IsValid(const string& strEGN)
{
    if (strEGN.length() != 10)

```

```

        return false;
    for (size_t i = 0; i < strEGN.length(); i++)
        if (!isdigit(static_cast<unsigned char>(strEGN[i])))
            return false;

    int nYear = (strEGN[0] - '0') * 10 + (strEGN[1] - '0');
    int nMonth = (strEGN[2] - '0') * 10 + (strEGN[3] - '0');
    int nDay = (strEGN[4] - '0') * 10 + (strEGN[5] - '0');

    if (nMonth >= 1 && nMonth <= 12)        nYear += 1900;
    else if (nMonth >= 21 && nMonth <= 32) { nYear += 1800; nMonth -= 20; }
    else if (nMonth >= 41 && nMonth <= 52) { nYear += 2000; nMonth -= 40; }
    else return false;

    int arrDays[12] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
    bool bLeap = (nYear % 4 == 0 && nYear % 100 != 0) || (nYear % 400 == 0);
    if (nMonth == 2 && bLeap) arrDays[1] = 29;
    if (nDay < 1 || nDay > arrDays[nMonth - 1]) return false;

    const int arrWeights[9] = { 2, 4, 8, 5, 10, 9, 7, 3, 6 };
    int nSum = 0;
    for (int i = 0; i < 9; i++)
        nSum += (strEGN[i] - '0') * arrWeights[i];

    int nCheck = nSum % 11;
    if (nCheck == 10) nCheck = 0;
    return nCheck == (strEGN[9] - '0');
}

// ----- Извеждане -----
ostream& CPerson::Output(ostream& toStream) const
{
    toStream << m_strName << " (" << genderToString() << "), ЕГН: "
        << (m_strEGN.empty() ? string("няма") : m_strEGN);
    return toStream;
}

// =====
// I. Клас CAddress
// =====
class CAddress
{
// 1-3. Частни членове
private:
    string m_strStreet;        // 1. улица, напр. "Studentska #1"
    string m_strPostCode;      // 2. пощенски код, напр. "9010"
    string m_strCity;          // 3. град, напр. "Varna"

public:
    // 1. Конструктор по подразбиране
    CAddress();
    // 2. Експлицитен конструктор
    CAddress(const string& strStreet, const string& strPostCode, const string& strCity);
    // 3. Копиращ конструктор
    CAddress(const CAddress& addr);

    // Функции за четене / презапис
    const string& GetStreet() const;
    const string& GetPostCode() const;

```

```

    const string& GetCity() const;
    void SetStreet(const string& strStreet);
    void SetPostCode(const string& strPostCode);
    void SetCity(const string& strCity);

    // 4. Функция за извеждане
    ostream& Output(ostream& toStream) const;
};

// ----- 1. Конструктор по подразбиране -----
CAddress::CAddress()
    : m_strStreet("(няма улица)"), m_strPostCode("0000"), m_strCity("(няма град)")
{
}

// ----- 2. Експлицитен конструктор -----
CAddress::CAddress(const string& strStreet, const string& strPostCode, const string& strCity)
    : m_strStreet(strStreet), m_strPostCode(strPostCode), m_strCity(strCity)
{
}

// ----- 3. Копиращ конструктор -----
CAddress::CAddress(const CAddress& addr)
    : m_strStreet(addr.m_strStreet),
      m_strPostCode(addr.m_strPostCode),
      m_strCity(addr.m_strCity)
{
}

// ----- Четене / презапис -----
const string& CAddress::GetStreet() const { return m_strStreet; }
const string& CAddress::GetPostCode() const { return m_strPostCode; }
const string& CAddress::GetCity() const { return m_strCity; }

void CAddress::SetStreet(const string& strStreet) { m_strStreet = strStreet; }
void CAddress::SetPostCode(const string& strPostCode) { m_strPostCode = strPostCode; }
void CAddress::SetCity(const string& strCity) { m_strCity = strCity; }

// ----- 4. Извеждане -----
// Връща потока, за да може да се вериги: addr.Output(cout) << endl;
ostream& CAddress::Output(ostream& toStream) const
{
    toStream << m_strCity << " " << m_strPostCode << ", " << m_strStreet;
    return toStream;
}

// =====
// II. Клас CStudent – наследник на CPerson
// =====
class CStudent : public CPerson
{
    // 1-2. Частни членове
private:
    string m_strFN; // 1. факултетен номер
    CAddress m_address; // 2. ВГРАДЕН обект от класа CAddress

public:
    // 1. Конструктор по подразбиране
    CStudent();

```

```

// 2. Експлицитни конструктори
CStudent(const string& strName, const CAddress& addr, const string& strEGN);
CStudent(const string& strName, const CAddress& addr, const string& strEGN,
        const string& strFN);
// 3. Копиращ конструктор
CStudent(const CStudent& stud);

// 5. Функции за четене / презапис
const string& GetFN() const;
const CAddress& GetAddress() const;
void SetFN(const string& strFN);
void SetAddress(const CAddress& addr);

// 4. Функция за извеждане
ostream& Output(ostream& toStream) const;
};

// ----- 1. Конструктор по подразбиране -----
// CPerson() и CAddress() се викат сами; списъкът е за яснота.
CStudent::CStudent()
    : CPerson(), m_strFN("(няма)"), m_address()
{
}

// ----- 2. Експлицитен конструктор по име, адрес и ЕГН -----
CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN)
    : CPerson(strName, "неопределен", strEGN), m_strFN("(няма)"), m_address(addr)
{
}

// ----- 2. Експлицитен конструктор по име, адрес, ЕГН и фак. номер -----
CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN,
        const string& strFN)
    : CPerson(strName, "неопределен", strEGN), m_strFN(strFN), m_address(addr)
{
}

// ----- 3. Копиращ конструктор -----
// CPerson(stud) копира наследената част, m_address(stud.m_address) – вградената.
CStudent::CStudent(const CStudent& stud)
    : CPerson(stud), m_strFN(stud.m_strFN), m_address(stud.m_address)
{
}

// ----- 5. Четене / презапис -----
const string& CStudent::GetFN() const { return m_strFN; }
const CAddress& CStudent::GetAddress() const { return m_address; }

void CStudent::SetFN(const string& strFN) { m_strFN = strFN; }
void CStudent::SetAddress(const CAddress& addr) { m_address = addr; }

// ----- 4. Извеждане -----
// Първо наследената част (CPerson::Output), после собствената.
ostream& CStudent::Output(ostream& toStream) const
{
    CPerson::Output(toStream);
    toStream << ", ф.н.: " << m_strFN << ", адрес: ";
    m_address.Output(toStream);
    return toStream;
}

```

```

}

// =====
// III. Главна функция
// =====
int main()
{
    // ===== A. Обекти от класа CAddress =====
    cout << "===== A. Класът CAddress =====" << endl;

    // а) трите конструктора
    CAddress addr1; // подразбиращ се
    CAddress addr2("Studentska #1", "9010", "Varna"); // експлицитен
    CAddress addr3(addr2); // копиращ

    cout << " addr1 (подразбиращ се): "; addr1.Output(cout) << endl;
    cout << " addr2 (експлицитен): "; addr2.Output(cout) << endl;
    cout << " addr3 (копие на addr2): "; addr3.Output(cout) << endl;

    // б) копието е независимо от оригинала
    cout << "---- Тест: копието е независимо ----" << endl;
    addr3.SetStreet("Studentska #2");
    cout << " addr2 след промяна на addr3: "; addr2.Output(cout) << endl;
    cout << " addr3 след промяна: "; addr3.Output(cout) << endl;

    // в) масив от обекти
    cout << "---- Масив от адреси ----" << endl;
    CAddress arrAddr[3] = {
        CAddress("Studentska #1", "9010", "Varna"),
        CAddress("Tsar Osvoboditel #12", "1000", "Sofia"),
        CAddress()
    };
    const int nAddrCount = sizeof(arrAddr) / sizeof(arrAddr[0]);
    for (int i = 0; i < nAddrCount; i++)
    {
        cout << " [" << i << "] ";
        arrAddr[i].Output(cout) << endl;
    }

    // г) указател в динамичната памет
    cout << "---- Указател (динамична памет) ----" << endl;
    CAddress* pAddr = new CAddress("Slivnitsa #33", "9002", "Varna");
    cout << " ";
    pAddr->Output(cout) << endl;
    delete pAddr;
    pAddr = nullptr;

    // д) референция
    cout << "---- Референция ----" << endl;
    CAddress& refAddr = addr1;
    refAddr.SetCity("Burgas");
    refAddr.SetPostCode("8000");
    cout << " addr1 след промяна през refAddr: "; addr1.Output(cout) << endl;

    // е) константен обект – само const функциите
    cout << "---- Константен обект ----" << endl;
    const CAddress cAddr("Studentska #1", "9010", "Varna");
    cout << " град: " << cAddr.GetCity() << ", код: " << cAddr.GetPostCode() << endl;
    // cAddr.SetCity("Ruse"); // грешка: обектът е константен

```

```

// ===== Б. Обекти от класа CStudent =====
cout << endl << "===== Б. Класът CStudent =====" << endl;

// а) четирите конструктора
CStudent stud1; // подразбира се
CStudent stud2("Иван Петров", addr2, "111111110"); // име, адрес, ЕГН
CStudent stud3("Елена Димитрова", addr2, "1111111104", "21621301");
CStudent stud4(stud3); // копиращ

cout << " stud1: "; stud1.Output(cout) << endl;
cout << " stud2: "; stud2.Output(cout) << endl;
cout << " stud3: "; stud3.Output(cout) << endl;
cout << " stud4: "; stud4.Output(cout) << endl;

// б) наследените функции работят върху наследника
cout << "---- Наследените функции на CPerson ----" << endl;
stud2.SetGender("мъж");
stud3.SetGender(FEMALE);
stud2.SetFN("21621300");
cout << " stud2: "; stud2.Output(cout) << endl;
cout << " stud3: "; stud3.Output(cout) << endl;
cout << " GetName() на stud3: " << stud3.GetName() << endl;
cout << " genderToString() на stud3: " << stud3.genderToString() << endl;

// в) копие е независимо
cout << "---- Тест: копие е независимо ----" << endl;
stud4.SetName("Друго име");
stud4.SetFN("00000000");
cout << " stud3 (оригинал): "; stud3.Output(cout) << endl;
cout << " stud4 (копие): "; stud4.Output(cout) << endl;

// г) масив от студенти
cout << "---- Масив от студенти ----" << endl;
CStudent arrStud[3] = {
    CStudent("Георги Стоянов", arrAddr[0], "111111110", "21621302"),
    CStudent("Мария Иванова", arrAddr[1], "1111111104", "21621303"),
    CStudent()
};
const int nStudCount = sizeof(arrStud) / sizeof(arrStud[0]);
for (int i = 0; i < nStudCount; i++)
{
    cout << " [" << i << "] ";
    arrStud[i].Output(cout) << endl;
}

// д) указател в динамичната памет
cout << "---- Указател (динамична памет) ----" << endl;
CStudent* pStud = new CStudent("Петър Колев", addr2, "111111110", "21621304");
cout << " ";
pStud->Output(cout) << endl;
delete pStud;
pStud = nullptr;

// е) указател към базовия клас сочи към наследник
cout << "---- Указател към базовия клас ----" << endl;
CPerson* pPerson = &stud3; // CStudent E CPerson
cout << " през CPerson: ";
pPerson->Output(cout) << endl; // вика CPerson::Output, НЕ CStudent::Output

```

```

// ж) референция
cout << "---- Референция ----" << endl;
CStudent& refStud = stud2;
refStud.SetAddress(CAddress("Slivnitsa #33", "9002", "Varna"));
cout << "   stud2 след промяна през refStud: "; stud2.Output(cout) << endl;

// з) константен обект
cout << "---- Константен обект ----" << endl;
const CStudent cStud("Анна Тодорова", addr2, "1111111104", "21621305");
cout << "   ";
cStud.Output(cout) << endl;
// cStud.SetFN("11111111");           // грешка: обектът е константен

// и) вграденият обект се чете през GetAddress()
cout << "---- Достъп до вградения обект ----" << endl;
cout << "   градът на cStud: " << cStud.GetAddress().GetCity() << endl;

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

Очакван изход

```

===== А. Класът CAddress =====
addr1 (подразбиращ се): (няма град) 0000, (няма улица)
addr2 (експлицитен):   Varna 9010, Studentska #1
addr3 (копие на addr2): Varna 9010, Studentska #1
--- Тест: копието е независимо ---
addr2 след промяна на addr3: Varna 9010, Studentska #1
addr3 след промяна:         Varna 9010, Studentska #2
--- Масив от адреси ---
[0] Varna 9010, Studentska #1
[1] Sofia 1000, Tsar Osvooboditel #12
[2] (няма град) 0000, (няма улица)
--- Указател (динамична памет) ---
Varna 9002, Slivnitsa #33
--- Референция ---
addr1 след промяна през refAddr: Burgas 8000, (няма улица)
--- Константен обект ---
град: Varna, код: 9010

===== Б. Класът CStudent =====
stud1: Неизвестен (неопределен), ЕГН: (няма), ф.н.: (няма), адрес: (няма град) 0000, (няма
улица)
stud2: Иван Петров (неопределен), ЕГН: 111111110, ф.н.: (няма), адрес: Varna 9010,
Studentska #1
stud3: Елена Димитрова (неопределен), ЕГН: 1111111104, ф.н.: 21621301, адрес: Varna 9010,
Studentska #1
stud4: Елена Димитрова (неопределен), ЕГН: 1111111104, ф.н.: 21621301, адрес: Varna 9010,
Studentska #1
--- Наследените функции на CPerson ---
stud2: Иван Петров (мъж), ЕГН: 111111110, ф.н.: 21621300, адрес: Varna 9010, Studentska #1
stud3: Елена Димитрова (жена), ЕГН: 1111111104, ф.н.: 21621301, адрес: Varna 9010, Studentska
#1

```

```

GetName() на stud3: Елена Димитрова
genderToString() на stud3: жена
--- Тест: копието е независимо ---
stud3 (оригинал): Елена Димитрова (жена), ЕГН: 1111111104, ф.н.: 21621301, адрес: Varna 9010, Studentska #1
stud4 (копие): Друго име (неопределен), ЕГН: 1111111104, ф.н.: 00000000, адрес: Varna 9010, Studentska #1
--- Масив от студенти ---
[0] Георги Стоянов (неопределен), ЕГН: 1111111110, ф.н.: 21621302, адрес: Varna 9010, Studentska #1
[1] Мария Иванова (неопределен), ЕГН: 1111111104, ф.н.: 21621303, адрес: Sofia 1000, Tsar Osvoboditel #12
[2] Неизвестен (неопределен), ЕГН: (няма), ф.н.: (няма), адрес: (няма град) 0000, (няма улица)
--- Указател (динамична памет) ---
Петър Колев (неопределен), ЕГН: 1111111110, ф.н.: 21621304, адрес: Varna 9010, Studentska #1
--- Указател към базовия клас ---
през CPerson*: Елена Димитрова (жена), ЕГН: 1111111104
--- Референция ---
stud2 след промяна през refStud: Иван Петров (мъж), ЕГН: 1111111110, ф.н.: 21621300, адрес: Varna 9002, Slivnitsa #33
--- Константен обект ---
Анна Тодорова (неопределен), ЕГН: 1111111104, ф.н.: 21621305, адрес: Varna 9010, Studentska #1
--- Достъп до вградения обект ---
градът на cStud: Varna

```

Три реда, които си струва да се разгледат.

stud4 (копие): Друго име (неопределен) — полът е „неопределен“, въпреки че stud3 вече е „жена“. Копието е направено *преди* извикването на SetGender(FEMALE) . Точно това доказва, че копието е независимо.

през CPerson*: ... ЕГН: 1111111104 и нищо повече — ранното свързване от 9.8.

addr1 след промяна през refAddr: Burgas 8000, (няма улица) — улицата е останала от конструктора по подразбиране, защото през референцията сменихме само град и код.

Чести грешки

Код	Съобщение	Причина и решение
C2248	cannot access private member declared in class 'CPerson'	Членовете на CPerson са private . Сменете на protected (стъпка 2).
C2247	'CPerson::SetName' not accessible because 'CStudent' uses 'private' to inherit	Пропуснатата дума public в class CStudent : public CPerson .
C2512	'CPerson': no appropriate default constructor available	Конструктор на наследника не вика базовия изрично, а базата няма конструктор по подразбиране.
C2652	illegal copy constructor: first parameter must not be 'CAddress'	Забравена е & в параметъра. Пише се const CAddress& addr .

C2668	ambiguous call to overloaded function	Два конструктора с еднакъв брой параметри от еднакви типове. Вижте бележката в стъпка 7.
C5038	member will be initialized after...	Предупреждение: редът в списъка за инициализация не съвпада с реда на обявяване.
C2143	syntax error: missing ';' before '}'	Липсва <code>;</code> след затварящата скоба на клас.
LNK2019	unresolved external symbol "public: __thiscall CAddress::CAddress(void)"	Конструкторът е обявен в класа, но тялото му не е написано. Проверете за печатна грешка в <code>CAddress::</code> .
LNK2005	main already defined	Файлт от тема 1 е в същия проект. Отделен проект за всяка задача.
—	Stack overflow при стартиране	<code>CStudent::Output</code> вика себе си. Пише се <code>CPerson::Output(toStream);</code> (стъпка 8).

Забележка към сигнатурата на `Output`

Условието изписва `ostream& Output(ostream& toStream)` — без `const` накрая. Ние пишем `... const`, защото функцията само чете данните на обекта. Разликата е практическа: без `const` не може да се изведе константен обект и редът

```
const CStudent cStud(...);
cStud.Output(cout);           // C2662, ако Output не е const
```

няма да се компилира. Ако преподавателят държи на буквалната сигнатура, махнете `const` от трите функции и премахнете тестовите с константни обекти от `main`.

Какво следва

Упражнение 3	Дълбоко копиране, деструктор, оператор <code>=</code> — какво се чути, когато класът държи динамична памет и се разчита на генерирания копиращ конструктор.
Упражнение 4	Приятелски функции и оператори: <code>Output</code> се заменя с <code>operator<<</code> , за да се пише <code>cout << stud;</code> .
Упражнение 5	Виртуални функции — решението на проблема от 9.8; и множествено наследяване.